

utfcode
utf8.sty 3.10 UTF-8 input encoding 13.06.2000
scanner for code UTF-8 installed.

Complete DSA Codes

Algorithm and manual Operations

Data Structures & Algorithms Lab

August 24, 2026

Contents

1	Lab 1 codes	3
2	Lab 2 codes	3
2.1	Array	3
2.1.1	Lab ₂	3
2.2	2d Array	5
2.2.1	Addition:	5
2.2.2	Subtraction	7
2.2.3	Multiplication	9
2.3	Linear Search	10
2.3.1	Using C++	10
2.3.2	Using C	11
2.3.3	Using Vector	12
2.4	Binary Search	13
2.4.1	Using C++	13
2.4.2	Using C	14
2.4.3	Using Vector	16
3	String	17
3.1	String Concatenation	17
3.1.1	Using C	17
3.1.2	Using C++	18
3.2	String Comparing	19
3.2.1	Using C	19
3.2.2	Using C++	20
3.3	String COpY	21
3.3.1	Using C	21
3.3.2	Using C++	22
3.4	String: Substring & Pattern Matching	23

1 Lab 1 codes

2 Lab 2 codes

2.1 Array

2.1.1 Lab₂

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 void arrTraverse(int arr[], int n) {
5     cout << "\nThe elements of the given array: ";
6     for (int i = 0; i < n; i++) {
7         cout << arr[i] << " ";
8     }
9     cout << endl;
10 }
11
12 void maxEl(int arr[], int n) {
13     int m = INT_MIN;
14     for (int i = 0; i < n; i++) {
15         if (m < arr[i]) m = arr[i];
16     }
17     cout << "The largest element of the array is: " << m << endl;
18 }
19
20 void minEL(int arr[], int n) {
21     int m = INT_MAX;
22     for (int i = 0; i < n; i++) {
23         if (m > arr[i]) m = arr[i];
24     }
25     cout << "The lowest element of the array is: " << m << endl;
26 }
27
28 int sumArray(int arr[], int n) {
29     int sum = 0;
30     for (int i = 0; i < n; i++) {
31         sum += arr[i];
32     }
33     cout << "The sum of the array: " << sum << endl;
34     return sum;
35 }
36
37 double avgArr(int sum, int n) {
38     double avg = (double)sum / n;
39     cout << "The average of the array is: " << avg << endl;
40     return avg;
41 }
42
43 void insertEl(int arr[], int &n, int val, int pos) {
44     if (pos < 0 || pos > n) {
45         cout << "Invalid insertion position!" << endl;
46         return;
47     }
48     for (int i = n; i > pos; i--) {
49         arr[i] = arr[i - 1];
50     }
51     arr[pos] = val;
```

```

52     n++;
53     cout << "Successfully inserted " << val << " at index " << pos << "." << endl;
54 }
55
56 void deleteEl(int arr[], int &n, int pos) {
57     if (pos < 0 || pos >= n) {
58         cout << "Invalid deletion position!" << endl;
59         return;
60     }
61     int removed = arr[pos];
62     for (int i = pos; i < n - 1; i++) {
63         arr[i] = arr[i + 1];
64     }
65     n--;
66     cout << "Successfully deleted element " << removed << " from index " << pos << "." << endl;
67 }
68
69 // Delete by value search
70 void deleteByValue(int arr[], int &n, int val) {
71     int pos = -1;
72     for (int i = 0; i < n; i++) {
73         if (arr[i] == val) {
74             pos = i;
75             break;
76         }
77     }
78
79     if (pos == -1) {
80         cout << "Element " << val << " not found in the array!" << endl;
81         return;
82     }
83
84     for (int i = pos; i < n - 1; i++) {
85         arr[i] = arr[i + 1];
86     }
87     n--;
88     cout << "Successfully deleted element " << val << " from index " << pos << "." << endl;
89 }
90
91 int main() {
92     int n;
93     cout << "Enter the size of an array: ";
94     cin >> n;
95
96     int arr[100];
97
98     cout << "Enter the elements: ";
99     for (int i = 0; i < n; i++) {
100         cin >> arr[i];
101     }
102
103     arrTraverse(arr, n);
104     maxEl(arr, n);
105     minEL(arr, n);
106     int sum = sumArray(arr, n);
107     avgArr(sum, n);
108

```

```

109 // Insertion
110 int valToInsert, insertPos;
111 cout << "\nEnter value to insert and target index (0-based): ";
112 cin >> valToInsert >> insertPos;
113 insertEl(arr, n, valToInsert, insertPos);
114 arrTraverse(arr, n);
115
116 // Deletion by Index
117 int deletePos;
118 cout << "\nEnter target index to delete (0-based): ";
119 cin >> deletePos;
120 deleteEl(arr, n, deletePos);
121 arrTraverse(arr, n);
122
123 // Deletion by Value
124 int valToDelete;
125 cout << "\nEnter element value to delete: ";
126 cin >> valToDelete;
127 deleteByValue(arr, n, valToDelete);
128 arrTraverse(arr, n);
129
130 return 0;
131 }

```

2.2 2d Array

2.2.1 Addition:

```

1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 // Function to add two dynamic 2D matrices
6 void addMat(int **a, int **b, int **res, int r, int c)
7 {
8     for (int i = 0; i < r; i++)
9     {
10         for (int j = 0; j < c; j++)
11         {
12             res[i][j] = a[i][j] + b[i][j];
13         }
14     }
15 }
16
17 // Function to print the matrix
18 void printRes(int **res, int r, int c)
19 {
20     cout << "\nResultant Matrix:\n";
21     for (int i = 0; i < r; i++)
22     {
23         for (int j = 0; j < c; j++)
24         {
25             cout << setw(4) << res[i][j] << " ";
26         }
27         cout << endl;
28     }
29 }
30
31 int main()

```

```

32 {
33     int r, c;
34     cout << "Enter the rows and cols of the matrix: ";
35     cin >> r >> c;
36
37     // Allocate rows (array of pointers)
38     int **a = new int *[r];
39     int **b = new int *[r];
40     int **res = new int *[r];
41
42     // Allocate columns for each row
43     for (int i = 0; i < r; i++)
44     {
45         a[i] = new int[c];
46         b[i] = new int[c];
47         res[i] = new int[c];
48     }
49
50     cout << "Enter the elements of array a:\n";
51     for (int i = 0; i < r; i++)
52     {
53         for (int j = 0; j < c; j++)
54         {
55             cout << "Enter the element [" << i << "][" << j << "]: ";
56             cin >> a[i][j];
57         }
58     }
59
60     cout << "The elements of array a:\n";
61     for (int i = 0; i < r; i++)
62     {
63         for (int j = 0; j < c; j++)
64         {
65             cout << a[i][j] << " ";
66         }
67         cout << endl;
68     }
69
70     cout << "Enter the elements of array b:\n";
71     for (int i = 0; i < r; i++)
72     {
73         for (int j = 0; j < c; j++)
74         {
75             cout << "Enter the element [" << i << "][" << j << "]: ";
76             cin >> b[i][j];
77         }
78     }
79
80     cout << "The elements of array b:\n";
81     for (int i = 0; i < r; i++)
82     {
83         for (int j = 0; j < c; j++)
84         {
85             cout << b[i][j] << " ";
86         }
87         cout << endl;
88     }
89
90     addMat(a, b, res, r, c);
91     printRes(res, r, c);

```

```

92
93     for (int i = 0; i < r; i++)
94     {
95         delete[] a[i];
96         delete[] b[i];
97         delete[] res[i];
98     }
99     delete[] a;
100    delete[] b;
101    delete[] res;
102
103    return 0;
104 }

```

2.2.2 Subtraction

```

1  #include <iostream>
2  #include <iomanip>
3  using namespace std;
4
5  // Function to add two dynamic 2D matrices
6  void subMat(int **a, int **b, int **res, int r, int c)
7  {
8      for (int i = 0; i < r; i++)
9      {
10         for (int j = 0; j < c; j++)
11         {
12             res[i][j] = a[i][j] - b[i][j];
13         }
14     }
15 }
16
17 // Function to print the matrix
18 void printRes(int **res, int r, int c)
19 {
20     cout << "\nResultant Matrix:\n";
21     for (int i = 0; i < r; i++)
22     {
23         for (int j = 0; j < c; j++)
24         {
25             cout << setw(4) << res[i][j] << " ";
26         }
27         cout << endl;
28     }
29 }
30
31 int main()
32 {
33     int r, c;
34     cout << "Enter the rows and cols of the matrix: ";
35     cin >> r >> c;
36
37     // Allocate rows (array of pointers)
38     int **a = new int *[r];
39     int **b = new int *[r];
40     int **res = new int *[r];
41
42     // Allocate columns for each row
43     for (int i = 0; i < r; i++)

```

```

44 {
45     a[i] = new int[c];
46     b[i] = new int[c];
47     res[i] = new int[c];
48 }
49
50 cout << "Enter the elements of array a:\n";
51 for (int i = 0; i < r; i++)
52 {
53     for (int j = 0; j < c; j++)
54     {
55         cout << "Enter the element [" << i << "][" << j << "]: ";
56         cin >> a[i][j];
57     }
58 }
59
60 cout << "The elements of array a:\n";
61 for (int i = 0; i < r; i++)
62 {
63     for (int j = 0; j < c; j++)
64     {
65         cout << a[i][j] << " ";
66     }
67     cout << endl;
68 }
69
70 cout << "Enter the elements of array b:\n";
71 for (int i = 0; i < r; i++)
72 {
73     for (int j = 0; j < c; j++)
74     {
75         cout << "Enter the element [" << i << "][" << j << "]: ";
76         cin >> b[i][j];
77     }
78 }
79
80 cout << "The elements of array b:\n";
81 for (int i = 0; i < r; i++)
82 {
83     for (int j = 0; j < c; j++)
84     {
85         cout << b[i][j] << " ";
86     }
87     cout << endl;
88 }
89
90 subMat(a, b, res, r, c);
91 printRes(res, r, c);
92
93 for (int i = 0; i < r; i++)
94 {
95     delete[] a[i];
96     delete[] b[i];
97     delete[] res[i];
98 }
99 delete[] a;
100 delete[] b;
101 delete[] res;
102
103 return 0;

```

2.2.3 Multiplication

```

1 #include <iostream>
2 #include <iomanip>
3 using namespace std;
4
5 void multMat(int **a, int **b, int **res, int r, int c)
6 {
7     for (int i = 0; i < r; i++)
8     {
9         for (int j = 0; j < c; j++)
10        {
11            res[i][j] = a[i][j] * b[i][j];
12        }
13    }
14 }
15
16 void printRes(int **res, int r, int c)
17 {
18     cout << "\nResultant Matrix:\n";
19     for (int i = 0; i < r; i++)
20     {
21         for (int j = 0; j < c; j++)
22         {
23             cout << setw(4) << res[i][j] << " ";
24         }
25         cout << endl;
26     }
27 }
28
29 int main()
30 {
31     int r, c;
32     cout << "Enter the rows and cols of the matrix: ";
33     cin >> r >> c;
34
35     // Allocate rows (array of pointers)
36     int **a = new int *[r];
37     int **b = new int *[r];
38     int **res = new int *[r];
39
40     // Allocate columns for each row
41     for (int i = 0; i < r; i++)
42     {
43         a[i] = new int[c];
44         b[i] = new int[c];
45         res[i] = new int[c];
46     }
47
48     cout << "Enter the elements of array a:\n";
49     for (int i = 0; i < r; i++)
50     {
51         for (int j = 0; j < c; j++)
52         {
53             cout << "Enter the element [" << i << "][" << j << "]: ";
54             cin >> a[i][j];
55         }

```

```

56     }
57
58     cout << "The elements of array a:\n";
59     for (int i = 0; i < r; i++)
60     {
61         for (int j = 0; j < c; j++)
62         {
63             cout << a[i][j] << " ";
64         }
65         cout << endl;
66     }
67
68     cout << "Enter the elements of array b:\n";
69     for (int i = 0; i < r; i++)
70     {
71         for (int j = 0; j < c; j++)
72         {
73             cout << "Enter the element [" << i << "][" << j << "]: ";
74             cin >> b[i][j];
75         }
76     }
77
78     cout << "The elements of array b:\n";
79     for (int i = 0; i < r; i++)
80     {
81         for (int j = 0; j < c; j++)
82         {
83             cout << b[i][j] << " ";
84         }
85         cout << endl;
86     }
87
88     multMat(a, b, res, r, c);
89     printRes(res, r, c);
90
91     for (int i = 0; i < r; i++)
92     {
93         delete[] a[i];
94         delete[] b[i];
95         delete[] res[i];
96     }
97     delete[] a;
98     delete[] b;
99     delete[] res;
100
101     return 0;
102 }

```

2.3 Linear Search

2.3.1 Using C++

Manual Operation

```

1 #include <iostream>
2 using namespace std;
3
4 int linearSearch(int arr[], int n, int target) {
5     for (int i = 0; i < n; i++) {
6         if (arr[i] == target) {

```

```

7         return i; // Target found at index i
8     }
9 }
10 return -1; // Target not found
11 }
12
13 int main() {
14     int arr[] = {12, 45, 23, 6, 89, 34};
15     int n = sizeof(arr) / sizeof(arr[0]);
16     int target = 23;
17
18     int index = linearSearch(arr, n, target);
19
20     if (index != -1)
21         cout << "Element " << target << " found at index " << index << endl;
22     else
23         cout << "Element " << target << " not found" << endl;
24
25     return 0;
26 }

```

Using STL

```

1 #include <iostream>
2 #include <algorithm>
3 #include <iterator>
4 using namespace std;
5
6 int main() {
7     int arr[] = {12, 45, 23, 6, 89, 34};
8     int n = sizeof(arr) / sizeof(arr[0]);
9     int target = 23;
10
11     // std::find returns an iterator pointing to the first match
12     auto it = find(arr, arr + n, target);
13
14     if (it != arr + n) {
15         int index = distance(arr, it); // Calculate index from iterator
16         cout << "Element " << target << " found at index " << index << endl;
17     } else {
18         cout << "Element " << target << " not found" << endl;
19     }
20
21     return 0;
22 }

```

2.3.2 Using C

Manual Operation

```

1 #include <stdio.h>
2
3 int linearSearch(int arr[], int n, int target) {
4     for (int i = 0; i < n; i++) {
5         if (arr[i] == target) {
6             return i; // Target found at index i
7         }
8     }
9     return -1; // Target not found
10 }
11

```

```

12 int main() {
13     int arr[] = {12, 45, 23, 6, 89, 34};
14     int n = sizeof(arr) / sizeof(arr[0]);
15     int target = 23;
16
17     int index = linearSearch(arr, n, target);
18
19     if (index != -1)
20         printf("Element %d found at index %d\n", target, index);
21     else
22         printf("Element %d not found\n", target);
23
24     return 0;
25 }

```

Using Standard Library

```

1 #include <stdio.h>
2 #include <search.h>
3
4 // Comparator callback function required by lfind
5 int compare(const void *a, const void *b) {
6     return (*(int *)a - *(int *)b);
7 }
8
9 int main() {
10     int arr[] = {12, 45, 23, 6, 89, 34};
11     size_t n = sizeof(arr) / sizeof(arr[0]);
12     int target = 23;
13
14     // lfind searches linearly for key match
15     int *result = (int *)lfind(&target, arr, &n, sizeof(int), compare);
16
17     if (result != NULL) {
18         int index = result - arr; // Pointer arithmetic to get index
19         printf("Element %d found at index %d\n", target, index);
20     } else {
21         printf("Element %d not found\n", target);
22     }
23
24     return 0;
25 }

```

2.3.3 Using Vector

Manually

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 int linearSearchVector(const vector<int>& vec, int target) {
6     for (size_t i = 0; i < vec.size(); i++) {
7         if (vec[i] == target) {
8             return i; // Target found at index i
9         }
10    }
11    return -1; // Target not found
12 }
13
14 int main() {

```

```

15     vector<int> vec = {12, 45, 23, 6, 89, 34};
16     int target = 23;
17
18     int index = linearSearchVector(vec, target);
19
20     if (index != -1)
21         cout << "Element " << target << " found at index " << index << endl;
22     else
23         cout << "Element " << target << " not found" << endl;
24
25     return 0;
26 }

```

Using Standard Library

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm> //find
4 #include <iterator> //distance
5 using namespace std;
6
7 int main() {
8     vector<int> vec = {12, 45, 23, 6, 89, 34};
9     int target = 23;
10
11     // std::find returns an iterator pointing to the matching element
12     auto it = find(vec.begin(), vec.end(), target);
13
14     if (it != vec.end()) {
15         int index = distance(vec.begin(), it); // Calculate index
16         cout << "Element " << target << " found at index " << index << endl;
17     } else {
18         cout << "Element " << target << " not found" << endl;
19     }
20
21     return 0;
22 }

```

2.4 Binary Search

2.4.1 Using C++

Manual Operation

```

1 #include <iostream>
2 #include <algorithm>
3 using namespace std;
4
5 int binarySearch(int arr[], int n, int target) {
6     int low = 0, high = n - 1;
7
8     while (low <= high) {
9         int mid = low + (high - low) / 2; // Prevents potential integer
10         overflow
11
12         if (arr[mid] == target)
13             return mid;
14         if (arr[mid] < target)
15             low = mid + 1;
16         else
17             high = mid - 1;
18     }
19 }

```

```

17     }
18     return -1; // Target not found
19 }
20
21 int main() {
22     int arr[] = {6, 12, 23, 34, 45, 89}; // Array must be sorted
23     int n = sizeof(arr) / sizeof(arr[0]);
24     int target = 34;
25
26     int index = binarySearch(arr, n, target);
27
28     if (index != -1)
29         cout << "Element " << target << " found at index " << index << endl;
30     else
31         cout << "Element " << target << " not found" << endl;
32
33     return 0;
34 }

```

Using Standard Library

```

1 #include <iostream>
2 #include <algorithm>
3 #include <iterator>
4 using namespace std;
5
6 int main() {
7     int arr[] = {6, 12, 23, 34, 45, 89};
8     int n = sizeof(arr) / sizeof(arr[0]);
9     int target = 34;
10
11     // 1. Quick check if element exists (returns bool)
12     bool exists = binary_search(arr, arr + n, target);
13
14     // 2. Get exact index using std::lower_bound
15     auto it = lower_bound(arr, arr + n, target);
16
17     if (it != arr + n && *it == target) {
18         int index = distance(arr, it);
19         cout << "Element " << target << " found at index " << index << endl;
20     } else {
21         cout << "Element " << target << " not found" << endl;
22     }
23
24     return 0;
25 }

```

2.4.2 Using C

Manual Operation

```

1 #include <stdio.h>
2
3 int binarySearch(int arr[], int n, int target) {
4     int low = 0, high = n - 1;
5
6     while (low <= high) {
7         int mid = low + (high - low) / 2;
8
9         if (arr[mid] == target)
10             return mid;

```

```

11         if (arr[mid] < target)
12             low = mid + 1;
13         else
14             high = mid - 1;
15     }
16     return -1;
17 }
18
19 int main() {
20     int arr[] = {6, 12, 23, 34, 45, 89};
21     int n = sizeof(arr) / sizeof(arr[0]);
22     int target = 34;
23
24     int index = binarySearch(arr, n, target);
25
26     if (index != -1)
27         printf("Element %d found at index %d\n", target, index);
28     else
29         printf("Element %d not found\n", target);
30
31     return 0;
32 }

```

Using Standard Library (bsearch)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 // Comparator callback function required by bsearch
5 int compare(const void *a, const void *b) {
6     return (*(int *)a - *(int *)b);
7 }
8
9 int main() {
10     int arr[] = {6, 12, 23, 34, 45, 89};
11     size_t n = sizeof(arr) / sizeof(arr[0]);
12     int target = 34;
13
14     // bsearch returns a pointer to the matching element
15     int *result = (int *)bsearch(&target, arr, n, sizeof(int), compare);
16
17     if (result != NULL) {
18         int index = result - arr; // Pointer arithmetic to get index
19         printf("Element %d found at index %d\n", target, index);
20     } else {
21         printf("Element %d not found\n", target);
22     }
23
24     return 0;
25 }
```

2.4.3 Using Vector

The vector must be sorted before performing binary search. Manual Implementation (Iterative)

```
1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4 using namespace std;
5
6 int binarySearchVector(const vector<int>& vec, int target) {
7     int low = 0;
8     int high = vec.size() - 1;
9
10    while (low <= high) {
11        int mid = low + (high - low) / 2;
12
13        if (vec[mid] == target)
14            return mid;
15        if (vec[mid] < target)
16            low = mid + 1;
17        else
18            high = mid - 1;
19    }
20    return -1; // Target not found
21 }
22
23 int main() {
24     vector<int> vec = {6, 12, 23, 34, 45, 89}; // Sorted vector
25     int target = 34;
26
27     int index = binarySearchVector(vec, target);
28
29     if (index != -1)
```

```

30     cout << "Element " << target << " found at index " << index << endl;
31     else
32         cout << "Element " << target << " not found" << endl;
33
34     return 0;
35 }

```

Using Standard Library (std::binary_search & std::lower_bound)

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4  #include <iterator>
5  using namespace std;
6
7  int main() {
8      vector<int> vec = {6, 12, 23, 34, 45, 89};
9      int target = 34;
10
11     // 1. Quick check for existence (returns true/false)
12     bool exists = binary_search(vec.begin(), vec.end(), target);
13
14     // 2. Get exact index using std::lower_bound
15     auto it = lower_bound(vec.begin(), vec.end(), target);
16
17     if (it != vec.end() && *it == target) {
18         int index = distance(vec.begin(), it);
19         cout << "Element " << target << " found at index " << index << endl;
20     } else {
21         cout << "Element " << target << " not found" << endl;
22     }
23
24     return 0;
25 }

```

3 String

3.1 String Concatenation

3.1.1 Using C

Manually

```

1  #include <stdio.h>
2
3  // Manual concatenation function
4  void manual_strcat(char *dest, const char *src) {
5      // 1. Move the dest pointer to the end of the destination string
6      while (*dest != '\0') {
7          dest++;
8      }
9
10     // 2. Copy characters from src to dest one by one
11     while (*src != '\0') {
12         *dest = *src;
13         dest++;
14         src++;
15     }
16
17     // 3. Manually add the null terminator at the end

```

```

18     *dest = '\\0';
19 }
20
21 int main() {
22     // Destination buffer must be large enough to hold both strings
23     char my_dest[50] = "Hello, ";
24     char my_src[] = "World!";
25
26     manual_strcat(my_dest, my_src);
27
28     printf("%s\\n", my_dest); // Outputs: Hello, World!
29     return 0;
30 }

```

Using strcat() or strncat() (Standard Library)

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     // dest must be explicitly sized to hold the combined data
6     char dest[50] = "Hello, ";
7     char src[] = "World!";
8
9     strcat(dest, src);
10    printf("%s\\n", dest); // Outputs: Hello, World!
11    return 0;
12 }

```

Using snprintf()

```

1 char dest[50];
2 char str1[] = "Hello";
3 char str2[] = "World";
4
5 // Formats and safely copies up to 50 bytes into dest
6 snprintf(dest, sizeof(dest), "%s, %s!", str1, str2);

```

3.1.2 Using C++

Manually

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Manual concatenation function using references
6 void manual_append(string &dest, const string &src) {
7     // Loop through each character of the source string
8     for (size_t i = 0; i < src.length(); i++) {
9         // Push each character to the end of the destination string
10        dest.push_back(src[i]);
11    }
12 }
13
14 int main() {
15     string str1 = "Hello, ";
16     string str2 = "World!";
17
18     manual_append(str1, str2);
19
20     cout << str1 << std::endl; // Outputs: Hello, World!

```

```

21     return 0;
22 }

```

1. Using the + or += Operators

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 int main() {
6     string s1 = "Hello";
7     string s2 = " World";
8
9     // Using + operator
10    string combined = s1 + s2;
11
12    // Using += operator (modifies s1 directly)
13    s1 += "!";
14
15    iostream << combined << s1; // Outputs: Hello WorldHello!
16    return 0;
17 }

```

2. Using the append() Member Function

```

1 string str = "Hello";
2 str.append(" World", 3); // Only appends the first 3 characters (" Wo")

```

3. Using std::ostringstream

```

1 #include <sstream>
2 #include <iostream>
3 using namespace std;
4
5 ostringstream ss;
6 ss << "User ID: " << 101 << " Name: " << "Alex";
7 string result = ss.str(); // Converts the stream back to a single string

```

3.2 String Comparing

3.2.1 Using C

Library Function

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char s1[] = "apple";
6     char s2[] = "apple";
7     char s3[] = "banana";
8
9     // Comparing identical strings
10    if (strcmp(s1, s2) == 0) {
11        printf("s1 and s2 are equal\n");
12    }
13
14    // Comparing different strings
15    if (strcmp(s1, s3) != 0) {
16        printf("s1 and s3 are not equal\n");
17    }
18    return 0;
19 }

```

Manually Without the Library

```
1 #include <stdio.h>
2
3 // Manual strcmp implementation
4 int manual_strcmp(const char *str1, const char *str2) {
5     // Loop as long as characters match and haven't hit the end of str1
6     while (*str1 && (*str1 == *str2)) {
7         str1++;
8         str2++;
9     }
10    // Return the difference of the mismatched characters
11    return *(unsigned char *)str1 - *(unsigned char *)str2;
12 }
13
14 int main() {
15     char s1[] = "apple";
16     char s2[] = "apricot";
17
18     int result = manual_strcmp(s1, s2);
19
20     if (result == 0) {
21         printf("Strings are equal\n");
22     } else if (result < 0) {
23         printf("s1 comes before s2\n"); // This will print ("p" < "r")
24     } else {
25         printf("s1 comes after s2\n");
26     }
27     return 0;
28 }
29
```

3.2.2 Using C++

1. Using Built-in C++ Library Operations

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     string s1 = "apple";
6     string s2 = "banana";
7
8     // Direct comparison using operators
9     if (s1 == s2) {
10         cout << "Strings are equal\n";
11     } else if (s1 < s2) {
12         cout << "s1 comes before s2 lexicographically\n";
13     }
14
15     // Using the built-in .compare() method (works like C's strcmp)
16     if (s1.compare(s2) == 0) {
17         cout << "Strings match\n";
18     }
19     return 0;
20 }
21
```

2. Manually Without String Comparison Functions

```
1 #include <iostream>
```

```

2 #include <string>
3 using namespace std;
4
5 // Manual comparison function returning true/false
6 bool manual_cpp_equal(const std::string &str1, const std::string &str2) {
7     // If lengths are different, strings cannot be equal
8     if (str1.length() != str2.length()) {
9         return false;
10    }
11
12    // Check every character index
13    for (size_t i = 0; i < str1.length(); ++i) {
14        if (str1[i] != str2[i]) {
15            return false; // Mismatch found
16        }
17    }
18    return true; // All characters matched
19 }
20
21 int main() {
22     string s1 = "apple";
23     string s2 = "apple";
24
25     if (manual_cpp_equal(s1, s2)) {
26         cout << "Strings are manually verified as equal\n";
27     }
28     return 0;
29 }
30

```

3.3 String COpy

3.3.1 Using C

1. Using the <string.h> Library

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char src[] = "Copy this text!";
6     char dest[30]; // Ensure buffer is large enough
7
8     // Using standard library copy
9     strcpy(dest, src);
10    printf("Library Copy: %s\n", dest);
11
12    // Safer bounded copy example
13    char safe_dest[10];
14    strncpy(safe_dest, src, sizeof(safe_dest) - 1);
15    safe_dest[sizeof(safe_dest) - 1] = '\0'; // Always manually null-
16    terminate
17    printf("Safe Bounded Copy: %s\n", safe_dest); // Outputs: Copy thi
18
19    return 0;
20 }

```

2. Manually Without the Library

```

1 #include <stdio.h>
2

```

```

3 // Manual string copy function
4 void manual_strcpy(char *dest, const char *src) {
5     // Loop through src until null terminator is found
6     while (*src != '\0') {
7         *dest = *src; // Assign character
8         dest++;      // Move dest pointer forward
9         src++;       // Move src pointer forward
10    }
11    *dest = '\0'; // Crucial: Explicitly null-terminate the destination
12    string
13 }
14 int main() {
15     char src[] = "Manual copy logic.";
16     char dest[30];
17
18     manual_strcpy(dest, src);
19     printf("Manual Copy: %s\n", dest);
20     return 0;
21 }

```

3.3.2 Using C++

1. Using Built-in C++ Library Operations

```

1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string src = "C++ Strings are easy.";
6
7     // Method A: Using the assignment operator (=)
8     std::string dest1;
9     dest1 = src; // Allocates memory and copies contents automatically
10
11    // Method B: Using the copy constructor
12    std::string dest2(src);
13
14    std::cout << "dest1: " << dest1 << "\ndest2: " << dest2 << std::endl;
15    return 0;
16 }

```

2. Manually Without String Assignment Operations

```

1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 // Manual string copy function in C++
6 void manual_cpp_copy(string &dest, const string &src) {
7     dest.clear(); // Empty out any existing text in destination
8
9     // Loop through each character of the source
10    for (char c : src) {
11        dest.push_back(c); // Append characters one by one
12    }
13 }
14
15 int main() {
16     string src = "Manually copying C++ Loop.";
17     string dest;

```

```

18     manual_cpp_copy(dest, src);
19     std::cout << "Manual C++ Copy: " << dest << std::endl;
20     return 0;
21 }
22

```

3.4 String: Substring & Pattern Matching

Manual:

```

1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  // Extract substring manually from start index for given length
6  string manualSubstring(const string& str, int start, int length) {
7      string result = "";
8      for (int i = start; i < start + length && i < str.length(); i++) {
9          result += str[i];
10     }
11     return result;
12 }
13
14 // Naive Pattern Matching (Returns starting index or -1)
15 int manualPatternMatch(const string& text, const string& pattern) {
16     int n = text.length();
17     int m = pattern.length();
18
19     for (int i = 0; i <= n - m; i++) {
20         int j;
21         for (j = 0; j < m; j++) {
22             if (text[i + j] != pattern[j])
23                 break;
24         }
25         if (j == m) return i; // Found match at index i
26     }
27     return -1; // Not found
28 }
29
30 int main() {
31     string text = "hello world";
32     string pattern = "world";
33
34     // Substring
35     string sub = manualSubstring(text, 6, 5);
36     cout << "Manual Substring: " << sub << endl;
37
38     // Pattern Matching
39     int pos = manualPatternMatch(text, pattern);
40     if (pos != -1)
41         cout << "Pattern found manually at index: " << pos << endl;
42     else
43         cout << "Pattern not found" << endl;
44
45     return 0;
46 }

```

Library:

```

1  #include <iostream>
2  #include <string>

```

```

3 using namespace std;
4
5 int main() {
6     string text = "hello world";
7     string pattern = "world";
8
9     // Substring using library: str.substr(pos, count)
10    string sub = text.substr(6, 5);
11    cout << "Library Substring: " << sub << endl;
12
13    // Pattern Matching using library: str.find(pattern)
14    size_t pos = text.find(pattern);
15
16    if (pos != string::npos)
17        cout << "Pattern found via find() at index: " << pos << endl;
18    else
19        cout << "Pattern not found" << endl;
20
21    return 0;
22 }

```